

1. What is a Web Application?

A **web application** is a software application that runs on a **web server** and is accessed through a **web browser** using the internet.

✚ Key Points:

- Provides **services over the web**
- Does **not require installation** on the client machine
- Accessible from **any device with a browser**
- Uses **client-server architecture**

✚ Examples:

- Gmail
- thekiranacademy.com
- Facebook
- Online Banking Applications

2. What are the types of Web Applications?

Web applications are mainly classified based on how they respond to user requests.

✚ Types:

1. **Static Web Applications**
2. **Dynamic Web Applications**

3. What is a Static Web Application?

A **static web application** returns the **same response** every time, regardless of user or time.

✚ Key Characteristics:

- Response **does not change**
- Content is **predefined**

- No database interaction
- Faster load time

Examples:

- Login Page of Gmail
 - Home Page of ICICI Bank
 - Portfolio websites
-

4. What is a Dynamic Web Application?

A **dynamic web application** generates **different responses** based on user, time, or data.

Key Characteristics:

- Response **varies from user to user**
- Uses **database**
- Requires **server-side programming**
- More interactive

Examples:

- Inbox Page of Gmail
 - Account Balance Page of ICICI Bank
 - E-commerce websites
-

5. Difference between Static and Dynamic Web Applications

Feature	Static Web App	Dynamic Web App
Response	Same for all users	Changes per user
Database	✗ Not used	✓ Used
Speed	Faster	Slightly slower
Interactivity	Low	High
Example	Home Page	User Dashboard

6. What is a Framework?

A **framework** is a **pre-built structure** that provides guidelines, tools, and libraries to develop applications efficiently.

✦ Key Points:

- Reduces **development time**
- Enforces **best practices**
- Provides **reusable code**
- Improves **maintainability**

✦ Examples:

- Django (Python)
 - Spring (Java)
 - React (JavaScript)
-

7. What is Django?

Django is a **free and open-source Python web framework** used to build secure and scalable web applications rapidly.

✦ Key Points:

- Written in **Python**
- Follows **MVT architecture**
- Maintained by **Django Software Foundation (DSF)**
- Batteries-included framework

🌐 Used By:

- YouTube
- Google
- Instagram
- NASA

8. Explain Django MVT Architecture

Django follows the **MVT (Model-View-Template)** architectural pattern.

✦ Components:

- **Model**
 - Represents **database tables** & Handles **data logic**
- **View**
 - Contains **business logic**
 - Processes requests and returns responses
- **Template**
 - Handles **frontend (HTML, CSS, JS)**
 - Displays data to users

✦ *Note:* Django itself manages the controller internally.

9. Explain the History of Django

Django was created to handle **high-traffic news websites** efficiently.

Timeline:

- **2003** – Developed as an internal project at *Lawrence Journal-World Newspaper*
- **Creators:**
 - Adrian Holovaty
 - Simon Willison
- **2005 (July 21)** – Released as **open-source**
- Named after guitarist **Django Reinhardt**

 Official Website: **djangoproject.com**

10. What are the Top 5 Features of Django?

Django is popular because of its powerful built-in features.

1. Fast

- Rapid development from idea to production
- Less boilerplate code

2. Fully Loaded

- Built-in:
 - Authentication
 - Admin panel
 - Forms
 - ORM
 - RSS & Sitemap support

3. Security

- Protection against:
 - SQL Injection
 - XSS
 - CSRF
 - Clickjacking
 - Secure password management
-

4. Scalability

- Handles **high traffic**
 - Used by large-scale applications
-

5. Versatile

- Used for:
 - CMS
 - Social networks
 - E-commerce
 - Scientific platforms

11. Difference Between Django and Flask

Django:

- Django is a **full-stack web framework**
- Comes with **built-in features**
- Best for **large & complex applications**

Flask:

- Flask is a **micro web framework**
- Very **lightweight and flexible**
- Best for **small apps & APIs**

Feature	Django	Flask
Type	Full-stack framework	Micro framework
Language	Python	Python
Architecture	MVT	MVC
Built-in Admin	✓ Yes	✗ No
ORM	✓ Django ORM	✗ Not built-in
Authentication	Built-in	External libraries
Scalability	High	Medium
Learning Curve	Moderate	Easy
Use Case	Large web apps	Small apps / APIs

12. Difference Between MVT and MVC

MVC (Model-View-Controller) :

- **Model** → Database & business logic
- **View** → UI (HTML, frontend)
- **Controller** → Handles requests & responses
- **Used In:** Spring, ASP.NET, Flask (loosely)

MVT (Model -View-Template)

- **Model** → Database tables
- **View** → Business logic
- **Template** → UI (HTML)
- **Used In:** Django

Feature	MVC	MVT
Full Form	Model-View-Controller	Model-View-Template
Used In	Flask, Spring	Django
Controller	Explicit	Handled internally
View Role	UI	Logic
Template Role	—	UI
Complexity	Higher	Simpler

13. What is MVT in Django?

MVT stands for **Model-View-Template**.

It is the **architectural pattern** used by **Django** to develop web applications.

- **Model** → Handles database
- **View** → Handles business logic
- **Template** → Handles UI (HTML)
- Django internally manages the **Controller**

“MVT separates data, logic, and presentation, which improves code readability and maintainability.”

14. Explain Model in Django MVT

The **Model** represents the **database structure** of the application.

✦ Responsibilities:

- Defines **database tables**
- Uses **Django ORM**
- Handles **CRUD operations**
- Maps Python classes to database tables

🔑 Example:

- User table
- Student table
- Result table

🎯 Interview Line:

“Model is responsible for interacting with the database using Django ORM.”

15. What is View in Django?

A **View** contains the **business logic** of the application.

✦ Responsibilities:

- Receives request from **URL**
- Interacts with **Model**
- Sends data to **Template**
- Returns **HTTP response**

🎯 Interview Line:

“View acts as the heart of Django where logic is written.”

16. What is Template in Django?

A **Template** is responsible for the **presentation layer**.

✦ Responsibilities:

- Displays data in **HTML**
- Uses **Django Template Language (DTL)**
- Separates UI from logic

🔗 Interview Line:

“Template handles only UI, not business logic.”

17. Explain the Flow of MVT Architecture

The MVT flow follows a structured request–response cycle.

✦ Steps:

1. User sends request
2. URL dispatcher maps URL
3. View processes logic
4. Model fetches data
5. Template renders HTML
6. Response sent to user

🔗 Interview Line:

“In Django, every request passes through URL → View → Model → Template.”

18. How is Controller handled in Django MVT?

Django **does not have an explicit controller**.

- **urls.py** works like a controller
- Django framework manages request routing internally
- “Django handles the controller part internally using URL dispatcher.”

19. Why Django Uses MVT Instead of MVC?

MVT provides **better separation of concerns**.

✦ Reasons:

- Cleaner code structure
- Faster development
- Built-in request handling
- Less boilerplate code

🎯 Interview Line:

“MVT helps Django reduce complexity and improve productivity.”

20. Real-Time Example of MVT

🔧 Example: Student Result System

- **Model** → Student, Result tables
- **View** → Calculate result, fetch marks
- **Template** → Display result page

🎯 Interview Line:

“In real applications, models store data, views process logic, and templates display results.”

21. Advantages of MVT Architecture

✦ Advantages:

- Separation of concerns
- Easy maintenance
- Reusable components
- Secure and scalable

🎯 Interview Line:

“MVT makes Django applications scalable and maintainable.”

22. What is the Django project folder structure?

The Django project folder structure is the **default directory layout** created when we run:

django-admin startproject projectname

It organizes **settings, URLs, apps, templates, and static files** in a structured way.

✦ Key Points:

- Separates **configuration and application logic**
- Improves **maintainability**
- Follows Django best practices

23. Explain the default Django project folder structure

📁 Example Structure:

projectname/

```
|
|— manage.py
|— projectname/
|   |— __init__.py
|   |— settings.py
|   |— urls.py
|   |— asgi.py
|   └— wsgi.py
```

manage.py

- Command-line utility
- Used to run server, migrations, createsuperuser

 Examples:

python manage.py runserver

python manage.py makemigrations

settings.py

- Main configuration file
- Database settings
- Installed apps
- Middleware
- Templates
- Static & media files

 Interview Line:

“settings.py controls the entire Django project configuration.”

urls.py

- Maps URLs to views
- Works like a **controller**

 Example:

`path('login/', views.login)`

__init__.py

- Marks directory as a Python package
- Usually empty

wsgi.py

- Used for **production deployment**
 - Handles synchronous requests
-

asgi.py

- Used for **asynchronous features**
 - Supports WebSockets
-

24. What is Django app folder structure?

An app is a **module inside a project** created using:

python manage.py startapp appname

25. Explain Django app folder structure

App Structure:

appname/

```
|
|
|— migrations/
|   └— __init__.py
|— admin.py
|— apps.py
|— models.py
|— tests.py
|— views.py
└— __init__.py
```

models.py

- Defines database tables
 - Uses Django ORM
-

views.py

- Contains business logic
 - Handles HTTP requests & responses
-

admin.py

- Registers models for Django admin panel
-

apps.py

- App configuration file
 - Contains app name and metadata
-

migrations/

- Stores database migration files
 - Tracks database changes
-

tests.py

- Used for unit testing
-

26. Where are templates stored in Django?

Templates can be stored in:

📁 Option 1: Project-level templates

projectname/

└─ templates/

📁 Option 2: App-level templates

appname/

└─ templates/

└─ appname/

📌 Recommended: **App-level templates**

27. Where are static files stored in Django?

Static files include CSS, JS, images.

📁 Structure:

static/

└─ css/

└─ js/

└─ images/

📌 Configured in settings.py

28. Difference between Project and App in Django

Project

Entire website

Contains settings

One project

App

One functionality

Contains logic

Multiple apps

 Interview Line:

“A project is a container and apps are reusable modules.”

29. Why Django uses multiple apps?

 Answer:

- Reusability
 - Clean code separation
 - Easy maintenance
 - Scalability
-

30. Advantages of Django Folder Structure

- Modular design
 - Easy debugging
 - Team-friendly
 - Industry standard
-

31. What is models.py in Django?

models.py is a file where we define **database models** in Django using **Python classes**.

✦ Key Points:

- Each model represents a **database table**
- Each attribute represents a **column**
- Django uses **ORM** to convert Python code into SQL

🎯 Interview Line:

“models.py defines the structure of the database using Django ORM.”

32. What is a Django Model?

A Django model is a **Python class** that inherits from `django.db.models.Model`.

✦ Key Points:

- Maps to a database table
- Handles CRUD operations
- Abstracts SQL queries

🔧 Example:

```
class Student(models.Model):  
  
    name = models.CharField(max_length=50)  
  
    age = models.IntegerField()
```

33. How does Django ORM work in models.py?

Django ORM converts **Python model code** into **SQL queries**.

✦ Key Points:

- No need to write raw SQL
- Database-independent
- Secure and efficient

🔗 Interview Line:

“Django ORM allows us to interact with the database using Python instead of SQL.”

34. What are Fields in Django Models?

Fields define the **type of data** stored in database columns.

✦ Common Fields:

- CharField
- IntegerField
- EmailField
- DateField
- BooleanField
- TextField

🔗 Interview Line:

“Model fields define column types and validations.”

35. What is primary_key in Django models?

A primary key uniquely identifies each record in a table.

- Django auto-creates id by default
- Can define custom primary key

```
id = models.AutoField(primary_key=True)
```

36. What is `__str__()` method in `models.py`?

`__str__()` defines **human-readable representation** of model objects.

✦ Purpose:

- Displays meaningful data in admin panel
- Improves readability

```
def __str__(self):
```

```
    return self.name
```

🎯 Interview Line:

“`__str__()` improves object representation in Django admin.”

37. What is Meta class in Django Models?

Meta class is used to define **model behavior**.

✦ Common Options:

- `db_table`
- `ordering`
- `verbose_name`
- `unique_together`

```
class Meta:
```

```
    db_table = 'student_table'
```

38. What are Relationships in Django Models?

Django supports **table relationships** using fields.

✦ Types:

- OneToOneField
- ForeignKey
- ManyToManyField

✎ Example:

```
class Result(models.Model):  
    student = models.ForeignKey(Student, on_delete=models.CASCADE)
```

39. What is on_delete in ForeignKey?

Defines behavior when referenced object is deleted.

✦ Options:

- CASCADE
- SET_NULL
- PROTECT
- DO_NOTHING

🎯 Interview Line:

“on_delete controls referential integrity.”

40. What are Migrations related to models.py?:

Migrations track **changes in models** and apply them to the database.

✦ Commands:

```
python manage.py makemigrations
```

```
python manage.py migrate
```

🔗 Interview Line:

“Migrations synchronize models with the database.”

41. What is blank vs null in models.py?

blank

Form-level validation

Used in forms

True/False

null

Database-level

Used in DB

True/False

🔗 Interview Line:

“blank is for validation, null is for database.”

42. What is unique=True in model fields?

Ensures **unique values** in a column.

```
email = models.EmailField(unique=True)
```

43. Real-Time Example of models.py

```
class Question(models.Model):  
  
    qno = models.IntegerField()  
  
    qtext = models.CharField(max_length=200)  
  
    subject = models.CharField(max_length=50)  
  
    answer = models.CharField(max_length=50)
```

44. Advantages of Using models.py

✦ Advantages:

- No SQL required
- Secure
- Database independent
- Easy maintenance

45. What is views.py in Django?

views.py contains the **business logic** of a Django application.
It handles **HTTP requests** and returns **HTTP responses**.

✦ Key Points:

- Acts as the **View layer** in MVT
- Receives request from urls.py
- Interacts with models.py
- Sends data to templates

🔗 Interview Line:

“views.py is where request processing and business logic are written.”

46. What are the types of Views in Django?

Django provides **two types of views**.

✦ Types:

1. **Function-Based Views (FBV)**
2. **Class-Based Views (CBV)**

47. What is a Function-Based View (FBV)?

A Function-Based View is a **Python function** that takes a request and returns a response.

✦ Key Points:

- Simple and easy to understand
- Good for small applications
- Uses `HttpResponse` or `render()`

✎ Example:

```
def home(request):  
    return HttpResponse("Welcome to Django")
```

🎯 Interview Line:

“FBVs are easy to write and best for simple logic.”

48. What is a Class-Based View (CBV)?

A Class-Based View uses **Python classes** to handle requests.

✦ Key Points:

- Reusable and organized
- Uses inheritance
- Suitable for large applications

✎ Example:

```
from django.views import View  
  
class HomeView(View):  
    def get(self, request):  
        return HttpResponse("Welcome")
```

🔗 Interview Line:

“CBVs improve code reusability and structure.”

49. Difference Between FBV and CBV

Feature	FBV	CBV
Type	Function	Class
Readability	Simple	Complex
Reusability	Low	High
Best For	Small apps	Large apps

50. What is HttpResponse in Django?

HttpResponse is used to return **plain text or HTML** response.

```
from django.http import HttpResponse  
  
return HttpResponse("Hello World")
```

✚ Used for:

- Simple responses
- API testing

51. What is render() in views.py?

render() combines a **template with data** and returns an HTTP response.

```
return render(request, "home.html", {"name": "TKA"})
```

✚ Advantages:

- Automatically loads template
- Sends context data

52. What is request object in views.py?

The request object contains **all client data**.

✚ Contains:

- request.method
- request.GET
- request.POST
- request.user

🎯 Interview Line:

“The request object carries user and form data.”

53. How does views.py interact with models.py?

Views fetch or store data using Django ORM.

```
students = Student.objects.all()
```

✧ Purpose:

- CURD operation

54. What is redirect() in views.py?

redirect() sends user to another URL.

```
return redirect("login")
```

✧ Used After:

- Form submission
- Login/logout

55. What are GET and POST methods in views.py?

- **GET** → Retrieve data
- **POST** → Submit data

```
if request.method == "POST":
```

```
    name = request.POST["name"]
```

56. What is JsonResponse in views.py?

Used to return **JSON data**, mainly in APIs.

```
from django.http import JsonResponse
```

```
return JsonResponse({"status": "success"})
```

57. Real-Time Example of views.py

Online Exam System:

```
def show_questions(request):
```

```
    questions = Question.objects.all()
```

```
    return render(request, "questions.html", {"questions": questions})
```

Flow:

- Fetch questions
- Send to template
- Display to user

58. Why views.py is important?

Reasons:

- Handles application logic
- Connects Model and Template
- Controls data flow

59. What is settings.py in Django?

settings.py is the **central configuration file** of a Django project. It controls how the project behaves.

Key Points:

- Project-level configuration
- Controls database, apps, templates, static files
- Loaded when Django server starts

Interview Line:

“settings.py defines the complete configuration of a Django project.”

60. What is INSTALLED_APPS in settings.py?

INSTALLED_APPS is a list of **enabled Django applications**.

- Default Django apps
- Third-party apps
- Custom apps

```
INSTALLED_APPS = [ 'django.contrib.admin',  
                   'django.contrib.auth',  
                   'students',]
```

“Every app must be registered in INSTALLED_APPS to work properly.”

61. What is DATABASES setting?

Defines **database configuration** used by Django.

✦ Common Databases:

- SQLite (default)
- MySQL
- PostgreSQL
- Oracle

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
    } }
```

🌀 Interview Line:

“DATABASES connects Django ORM with the actual database.”

62. What is MIDDLEWARE in Django?

Middleware is a **request–response processing layer**.

✦ Responsibilities:

- Security
- Sessions
- Authentication
- CSRF protection

🔗 Interview Line:

“Middleware acts like a filter between request and response.”

63. What is TEMPLATES setting?

Defines how Django finds and loads **HTML templates**.

✦ Includes:

- Template directories
- Context processors
- Template engine

```
TEMPLATES = [{ 'DIRS': [BASE_DIR / 'templates'], }]
```

64. What are STATIC_URL and STATICFILES_DIRS?

Used to manage **static files** like CSS, JS, images.

- STATIC_URL → URL path
- STATICFILES_DIRS → File location

```
STATIC_URL = '/static/'
```

```
STATICFILES_DIRS = [BASE_DIR / 'static']
```

65. What is MEDIA_URL and MEDIA_ROOT?

Used for **user-uploaded files**.

```
MEDIA_URL = '/media/'
```

```
MEDIA_ROOT = BASE_DIR / 'media'
```

✦ Example:

- Profile pictures
- Uploaded documents

66. What is DEBUG in settings.py?

Controls **debug mode**.

✦ Values:

- True → Development
- False → Production

🔗 Interview Line:

“DEBUG must be False in production for security reasons.”

67. What is ALLOWED_HOSTS?

Defines which **domains/IPs** can access the project.

```
ALLOWED_HOSTS = ['localhost', '127.0.0.1']
```

✦ Prevents HTTP Host header attacks.

68. What is SECRET_KEY in Django?

Used for **security-related operations**.

✦ Used For:

- Password hashing
- Session security
- CSRF tokens

🔗 Interview Line:

“SECRET_KEY should never be shared or committed to GitHub.”

69. What is ROOT_URLCONF?

Points to the **main URL configuration file**.

ROOT_URLCONF = 'projectname.urls'

70. What is WSGI_APPLICATION?

Used for **deployment** in synchronous servers.

WSGI_APPLICATION = 'projectname.wsgi.application'

71. What is ASGI_APPLICATION?

Used for **asynchronous support**.

✦ Supports:

- WebSockets
 - Async views
-

72. Real-Time Example of settings.py Usage

- Database → Student, Question tables
- Media → Profile images
- Static → CSS for exam UI
- Installed apps → users, exams, results

73. What is a Session in Django?

A session is a way to store user data on the server across multiple requests.

✦ Key Points:

- Stores data per user
- Data persists between requests
- Uses session ID stored in browser cookies
- More secure than cookies

🎯 Interview Line:

“Sessions allow Django to remember user data across multiple requests.”

74. Why are Sessions used in Django?

Sessions are used to maintain state in web applications.

✦ Uses:

- User login state
- Shopping cart data
- Exam progress
- Temporary user data

🎯 Interview Line:

“Sessions are used to track logged-in users and store temporary data.”

75. How does Django Session work internally?

Django uses a session ID to identify users.

✦ Flow:

1. User sends request
2. Django creates a session ID
3. Session ID stored in browser cookie
4. Actual data stored on server
5. Session retrieved on next request

Browser → Session ID → Server → Session Data

76. Where is Session data stored in Django?

✓ Answer:

Django supports multiple session storage backends.

✦ Types:

- Database (default)
- Cache
- File-based
- Cookies (less secure)

✦ Default Table:

django_session

77. How to enable Sessions in Django?

Sessions are enabled by default.

✦ Required in settings.py:

```
INSTALLED_APPS = [
```

```
    'django.contrib.sessions',
```

```
]
```

```
MIDDLEWARE = [
```

```
    'django.contrib.sessions.middleware.SessionMiddleware',
```

```
]
```

78. How to create a session in Django?

Sessions are created using the request.session object.

```
request.session['username'] = 'TKA'
```

✦ Stores data on server side.

79. How to read session data in Django?

Use the session key.

```
username = request.session.get('username')
```

✦ get() avoids KeyError.

80. How to delete session data in Django?

◆ Delete specific key:

```
del request.session['username']
```

◆ Clear all session data:

```
request.session.flush()
```

🔗 Interview Line:

“flush() removes session data and session ID.”

81. What is the difference between Sessions and Cookies?

Sessions

Stored on server

More secure

Stores large data

Slower

Cookies

Stored in browser

Less secure

Limited size

Faster

🔗 Interview Line:

“Sessions are more secure because data is stored on server.”

82. What is Session Expiry?

Session expiry defines how long a session is valid.

✦ Default:

- Expires when browser closes

✦ Custom Expiry:

```
request.session.set_expiry(300) # 5 minutes
```

83. What is SESSION_ENGINE?

Defines where session data is stored.

✦ Example:

`SESSION_ENGINE = 'django.contrib.sessions.backends.db'`

✦ Other options:

- cache
 - file
 - signed_cookies
-

84. What is SESSION_COOKIE_AGE?

Defines session lifetime in seconds.

`SESSION_COOKIE_AGE = 1200` # 20 minutes

85. Are Django Sessions secure?

Yes, Django sessions are secure by default.

✦ Security Features:

- Signed cookies
- CSRF protection
- Server-side storage

🎯 Interview Line:

“Django provides secure session management out of the box.”

86. What is CRUD in Django?

CRUD stands for Create, Read, Update, and Delete operations performed on database records using Django ORM.

✦ Key Points:

- Used to manage database data
- Implemented using models, views, and templates
- Django ORM handles SQL internally

🎯 Interview Line:

“CRUD operations allow us to insert, retrieve, modify, and remove data.”

87. How do you perform CREATE operation in Django?

Create operation inserts new records into the database.

✦ Methods:

1. `save()`
2. `objects.create()`

🔗 Example:

```
student = Student(name="Rahul", age=20)
```

```
student.save()
```

OR

```
Student.objects.create(name="Rahul", age=20)
```

88. How do you perform READ operation in Django?

Read operation retrieves data from the database.

✦ Common Methods:

- `all()`
- `get()`
- `filter()`

✎ Example:

```
students = Student.objects.all()
```

🔗 Interview Line:

“Read operation fetches records using ORM querysets.”

89. Difference between `get()` and `filter()`?

`get()`

Returns single object

Raises error if multiple

Used with unique fields

`filter()`

Returns queryset

No error

Used with conditions

90. How do you perform UPDATE operation in Django?

Update operation modifies existing records.

✦ Methods:

1. Using `save()`
2. Using `update()`

✎ Example:

```
student = Student.objects.get(id=1)
```

```
student.name = "Amit"
```

```
student.save()
```

OR

```
Student.objects.filter(id=1).update(name="Amit")
```

91. How do you perform DELETE operation in Django?

Delete operation removes records from database.

 **Example:**

```
student = Student.objects.get(id=1)
```

```
student.delete()
```

 **Interview Line:**

“Delete operation permanently removes data from database.”

92. What is Django ORM in CRUD operations?

Django ORM allows database operations using Python objects instead of SQL.

 **Advantages:**

- Database independent
 - Secure
 - Readable code
-

93. What is QuerySet in Django CRUD?

A QuerySet is a collection of database objects.

 **Key Points:**

- Lazy evaluation
- Can be filtered, sliced, ordered

```
Student.objects.filter(age__gte=18)
```

94. How do you display data in CRUD?

Data is passed from view to template using context dictionary.

```
return render(request, "html.html", {"students": students})
```

95. Real-Time Example of CRUD in Django

- **Create** → Add questions
 - **Read** → View questions
 - **Update** → Edit questions
 - **Delete** → Remove questions
-

96. Difference between save() and create()?

save()

Two-step process

Object then save

More control

create()

One-step process

Direct save

Faster

97. What is path()?

path() is used to define a URL pattern and directly connect it to a view function or class.

🔑 Key Points:

- Maps a URL to a view
 - Written inside urls.py
 - Handles individual routes
 - Used for simple URL mapping
-

Example :

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [  
    path('login/', views.login_view),  
    path('home/', views.home),  
]
```

"path() is used to connect a URL directly to a view."

98. What is include()?

include() is used to **include another URL configuration file** (usually app-level urls.py) into the main project urls.py.

✦ Key Points:

- Used for **URL modularization**
- Helps split large URL files
- Used in **project-level urls.py**
- Promotes **reusability and clean structure**

Example:

```
from django.urls import path, include
```

```
urlpatterns = [  
    path('accounts/', include('accounts.urls')),  
    path('exam/', include('exam.urls')),  
]
```

"include() is used to connect project URLs with app-level URLs."

Feature	path()	include()
Purpose	Map URL to view	Include another URL file
Used In	Project & app urls.py	Mostly project urls.py
Connects To	View function / CBV	App-level urls.py
Complexity	Simple	Modular
Best For	Small apps	Large projects
Reusability	☒ No	☐ Yes

99. What is MySQL connectivity in Django?

MySQL connectivity in Django means connecting a Django application to a MySQL database so that Django ORM can perform database operations.

✦ Key Points:

- Django supports MySQL officially
- Database configuration is done in settings.py
- Django ORM converts Python code into MySQL queries
- Used for production-level applications

🔗 Interview Line:

“Django connects to MySQL using database drivers and Django ORM.”

100. Which MySQL database driver is used in Django?

Django uses mysqlclient as the recommended MySQL database adapter.

✦ Key Points:

- mysqlclient is fast and stable
- Written in C (better performance)
- Required for Django–MySQL connectivity

✦ Installation:

pip install mysqlclient

🔗 Interview Line:

“mysqlclient is the recommended MySQL adapter for Django.”

101. How do you configure MySQL in Django?

MySQL is configured in the DATABASES setting inside settings.py.

🔗 Example:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.mysql',  
        'NAME': 'examdb',  
        'USER': 'root',  
        'PASSWORD': 'root',  
        'HOST': 'localhost',  
        'PORT': '3306',  
    }  
}
```

✦ Key Fields:

- ENGINE → MySQL backend
 - NAME → Database name
 - USER → MySQL username
 - PASSWORD → MySQL password
 - HOST → Server address
 - PORT → MySQL port
-

102. What is ORM?

ORM (Object Relational Mapping) is a technique that allows developers to interact with the database using programming language objects instead of SQL queries.

✦ Key Points:

- Converts Python objects ↔ Database tables
- Eliminates writing raw SQL
- Improves security and readability
- Database independent

🎯 Interview Line:

“ORM allows database operations using Python code instead of SQL.”

103. What is Django ORM?

Django ORM is a built-in ORM framework that enables database operations using Python classes and objects.

- Each model = database table
- Each field = table column
- Uses QuerySets to retrieve data
- Supports MySQL, PostgreSQL, SQLite, Oracle

🎯 Interview Line:

“Django ORM abstracts database queries into Python code.”

104. Why is ORM used in Django?

ORM is used to make database interaction simple, secure, and portable.

🔑 Advantages:

- No SQL required
 - Protects from SQL Injection
 - Faster development
 - Easy maintenance
 - Database independent
-

105. How does Django ORM work?

Django ORM converts Python model queries into SQL queries internally.

🔑 Example:

`Student.objects.all()`

→ Internally converted to:

`SELECT * FROM student;`

🔑 Flow:

Python Code → Django ORM → SQL → Database → Result

106. What is objects in Django ORM?

objects is the default manager provided by Django ORM.

✦ Used For:

- Querying database
- Creating, updating, deleting records

Student.objects.all()

107. Difference between ORM and Raw SQL

Feature	ORM	Raw SQL
Language	Python	SQL
Security	High	Risky
Readability	Easy	Complex
Portability	Database independent	DB specific
Speed	Slightly slower	Faster